

# Reporte de Pruebas Unitarias de la API Nexa

Cobertura de endpoints, validación de errores y resultados obtenidos

| Dato                | Valor                                              |
|---------------------|----------------------------------------------------|
| Actividad           | Pruebas unitarias de endpoints previamente creados |
| Tecnología          | Node.js 22, node:test, assert y SQLite             |
| Fecha de ejecución  | 1 de octubre de 2026                               |
| Resultado           | 17 pruebas aprobadas de 17; 0 fallos               |
| Cobertura funcional | 12 operaciones HTTP y protocolo Socket TCP 6061    |

## Resumen ejecutivo

Se implementó y ejecutó una suite automatizada sobre la API desarrollada en la práctica anterior. La prueba crea una base SQLite temporal y aislada, inicia el servidor en un puerto disponible y valida el contrato JSON **{statusCode, data}**. La ejecución cubrió lectura, creación, actualización, eliminación, respaldo, vaciado y comunicación TCP.

Para cumplir el requisito de actualización se incorporaron dos operaciones PUT: una para categorías y otra para productos. La documentación OpenAPI se actualizó a 12 operaciones. La suite también reproduce errores comunes del consumidor, como datos incompletos, identificadores inválidos, token incorrecto, JSON mal formado, tipo de contenido ausente, duplicados y relaciones inexistentes.

## Conclusión

El resultado fue satisfactorio: **17 de 17 pruebas aprobaron**. La API mantiene la estructura uniforme de respuesta, protege las escrituras, valida los cuerpos con JSON Schema, conserva la integridad referencial y genera respaldos SQLite coherentes.

## Objetivo y estrategia de prueba

El objetivo fue comprobar que los endpoints responden correctamente ante solicitudes válidas y rechazan de forma controlada las entradas incorrectas. Se utilizó el módulo estándar **node:test**, equivalente funcional a Jest para esta práctica, junto con **assert/strict**.

## Preparación del entorno

Cada ejecución crea una carpeta temporal, inicializa el esquema normalizado de categorías y productos, configura un token administrativo exclusivo de prueba y levanta servidores HTTP y TCP locales. Al finalizar se cierran los servidores y se elimina la base temporal. Así, las pruebas no alteran la base desplegada en EC2.

## Comando de ejecución

```
node --test --test-reporter spec test.mjs
```

## Criterios verificados

Cada respuesta HTTP debe incluir solamente las propiedades **statusCode** y **data**; data siempre debe ser un arreglo. También se compara el código HTTP real con statusCode, se revisan los datos devueltos y se confirma la persistencia después de reiniciar el servidor.

## Archivos principales

| Archivo       | Función                                        |
|---------------|------------------------------------------------|
| server.mjs    | Servidor HTTP, Socket TCP y acceso a SQLite    |
| contracts.mjs | JSON Schema, validador y documentación OpenAPI |
| test.mjs      | Suite automatizada de 17 pruebas               |
| schema.sql    | Tablas normalizadas, restricciones e índice    |

## Matriz de los 12 endpoints probados

La matriz incluye operaciones GET, POST, PUT y DELETE. PUT representa la actualización solicitada en la actividad.

| Núm. m. | Método | Ruta                 | Comportamiento esperado               | HTTP |
|---------|--------|----------------------|---------------------------------------|------|
| 1       | GET    | /api/health          | Estado del servicio y conexión SQLite | 200  |
| 2       | GET    | /api/categories      | Lista categorías                      | 200  |
| 3       | POST   | /api/categories      | Crea una categoría válida             | 201  |
| 4       | PUT    | /api/categories/{id} | Actualiza el nombre de categoría      | 200  |
| 5       | DELETE | /api/categories/{id} | Elimina categoría sin productos       | 200  |
| 6       | GET    | /api/products        | Lista productos                       | 200  |
| 7       | GET    | /api/products/{id}   | Consulta producto por identificador   | 200  |
| 8       | POST   | /api/products        | Crea producto relacionado             | 201  |
| 9       | PUT    | /api/products/{id}   | Actualiza nombre, precio y categoría  | 200  |
| 10      | DELETE | /api/products/{id}   | Elimina producto                      | 200  |
| 11      | POST   | /api/database/backup | Crea respaldo SQLite consistente      | 200  |
| 12      | DELETE | /api/database        | Vacía datos y conserva estructura     | 200  |

## Escenarios de fallo considerados

Los siguientes casos representan errores previsibles que un usuario puede cometer al consumir la API. Todos devuelven una respuesta JSON controlada y un código HTTP adecuado.

| Entrada o acción                 | Endpoint           | Código    | Validación                          |
|----------------------------------|--------------------|-----------|-------------------------------------|
| Body vacío, arreglo o null       | POST categoría     | 400       | El cuerpo no cumple el JSON Schema  |
| Nombre vacío o propiedad extra   | POST categoría     | 400       | Rechazo de entrada inválida         |
| Precio negativo, decimal o texto | POST producto      | 400       | Validación de tipo y rango          |
| Categoría inexistente            | POST producto      | 409       | Conflicto de integridad referencial |
| Categoría duplicada              | POST categoría     | 409       | Restricción UNIQUE                  |
| Eliminar categoría con productos | DELETE categoría   | 409       | Protección de la relación           |
| Token incorrecto o ausente       | POST PUT DELETE    | 401       | Acceso administrativo rechazado     |
| Confirmación distinta de VACIAR  | DELETE base        | 400       | Evita borrado accidental            |
| ID inexistente                   | GET o PUT producto | 404       | Registro no encontrado              |
| ID no numérico                   | GET producto       | 400       | Identificador inválido              |
| JSON mal formado                 | POST categoría     | 400       | JSON inválido                       |
| Sin application/json             | POST categoría     | 415       | Tipo de contenido requerido         |
| Body mayor a 16 KB               | POST categoría     | 413       | Cuerpo demasiado grande             |
| Ruta o método inexistente        | GET o PATCH        | 404 o 405 | Error controlado                    |

## Resultados obtenidos

| Métrica                       | Resultado                        |
|-------------------------------|----------------------------------|
| Pruebas ejecutadas            | 17                               |
| Aprobadas                     | 17                               |
| Fallidas                      | 0                                |
| Canceladas                    | 0                                |
| Omitidas                      | 0                                |
| Cobertura de operaciones HTTP | 12 endpoints                     |
| Prueba adicional              | Socket TCP 6061 con insert y get |

## Evidencia resumida de la ejecución

- ✓ GET health y documentación OpenAPI
- ✓ GET, POST, PUT y DELETE de categorías
- ✓ GET, POST, PUT y DELETE de productos
- ✓ Respaldo SQLite e integrity\_check = ok
- ✓ Persistencia después de reiniciar
- ✓ Vaciado conservando tablas y respaldos
- ✓ Errores de esquema, autorización e integridad
- ✓ Socket TCP 6061 insert y get

Resultado: 17 aprobadas, 0 fallidas

## Interpretación

La aprobación total demuestra que las funciones principales cumplen sus contratos y que las entradas inválidas no producen respuestas ambiguas ni corrupción de datos. El respaldo se abrió en modo de solo lectura, pasó PRAGMA integrity\_check y conservó el registro creado antes de la copia.

## Limitaciones

Estas pruebas validan comportamiento funcional e integración local con SQLite. No sustituyen pruebas de carga, seguridad ofensiva ni disponibilidad prolongada de EC2. El despliegue debe ejecutar la misma versión del código para disponer de los nuevos endpoints PUT.

## Recomendaciones

Ejecutar **npm test** antes de construir cada imagen Docker, agregar la prueba al flujo de integración continua y desplegar una nueva imagen en EC2 después de cualquier cambio en server.mjs o contracts.mjs.